Turning dataflow facts into assertions was conceptually pretty simple because I implemented the simple sign lattice. This meant that all assertions could just be asserting that some value was less than, equal to, or greater than 0 (or combinations for ZeroOrNeg and so on). Actually inserting the assertions was a little bit more complicated because dealing with the LLVM dialect of MLIR and the C++ library for it got a bit messy, but overall it wasn't too bad.

This is the important section of C++ logic for inserting tests:

```
// Create zero with the same type as value.
auto zeroOp = mlir::LLVM::ConstantOp::create(
    builder, location, value.getType(),
    builder.getIntegerAttr(value.getType(), 0));

// Comparison predicate
mlir::LLVM::ICmpPredicate predicate;
switch (kind) {
case Kind::Top:
case Kind::Bottom:
    return;
case Kind::Zero:
    predicate = mlir::LLVM::ICmpPredicate::eq;
    break;
case Kind::Negative:
    predicate = mlir::LLVM::ICmpPredicate::slt;
    break;
case Kind::ZeroOrNeg:
    predicate = mlir::LLVM::ICmpPredicate::sle;
    break;
case Kind::Positive:
    predicate = mlir::LLVM::ICmpPredicate::sgt;
    break;
case Kind::ZeroOrPos:
    predicate = mlir::LLVM::ICmpPredicate::sge;
    break;
}

mlir::LLVM::ICmpOp compareOp = mlir::LLVM::ICmpOp::create(
    builder, location, predicate, value, zeroOp.getResult());
```

The comparison produces a boolean condition. To enforce it, I split the block and insert `llvm.cond_br` to either continue or enter a failure block, which prints the failed check and executes `llvm.intr.trap` followed by `llvm.unreachable`.

I then generated a set of small MLIR tests, several of which exposed the same bug. The runner converts the instrumented MLIR back to LLVM IR, compiles it with Clang, and compares its execution with an uninstrumented baseline on the same inputs. This is one of the tests:

```
module {
  llvm.func @main(%argc: i32, %argv: !llvm.ptr) -> i32 {
    %zero = llvm.mlir.constant(0 : i32) : i32
    %one = llvm.mlir.constant(1 : i32) : i32
    %small = llvm.mlir.constant(5 : i8) : i8
    %maximum = llvm.mlir.constant(127 : i8) : i8
    %increment = llvm.mlir.constant(1 : i8) : i8
    %boundary = llvm.icmp "sgt" %argc, %one : i32
    llvm.cond_br %boundary, ^large, ^small
  ^large:
    llvm.br ^join(%maximum : i8)
  ^small:
    llvm.br ^join(%small : i8)
  ^join(%x: i8):
    %result = llvm.add %x, %increment : i8
    llvm.return %zero : i32
  }
}
```

Running this program with one extra argument makes `argc` equal to 2 and selects `%maximum`. The original analysis predicted Positive for `%result`, because both operands were Positive. But `127 + 1` wraps to `-128` in `i8`, so the inserted assertion failed with “expected: positive” and “actual: −128”. The addition has no overflow flags, so this wrapping behavior is defined in LLVM. With no extra arguments, the program instead computes `5 + 1` and the check passes.

The bug: I copied the sign lattice and arithmetic rules from the book, but those rules assumed mathematical integers and did not consider overflow. This bug was already in my repo unintentionally; I did not have to insert one. With overflow in the picture a lot of the transfer rules were incorrect. This is obvious in hindsight but it really shows the limitations of the sign analysis lattice.

With only sign information available, the fix was to make the potentially wrapping cases return Top. This cuts out a large core of the important pieces of info that this analysis is able to find. I changed the addition, subtraction, and multiplication tables for potentially wrapping cases involving positive or negative lattice elements. Cases such as adding zero or multiplying by zero can still give useful information. After the fix, all 18 test runs pass.

This really shows why sign analysis alone is limited for fixed-width arithmetic, and why range analysis or known bits can be more useful for tracing integers.